ARTICULACIONES EN GRAFOS

DFS, tiempos de descubrimiento y valores low

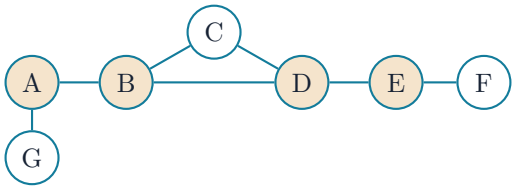
Antony Yonel Rosales Esteban | Jeseph Imanol Burgos Ochoa

Resumen. Un punto de articulación es un vértice cuya eliminación aumenta el número de componentes conexas. Presentamos una implementación original de DFS de Tarjan que calcula los tiempos tin y los valores low, produce una traza reproducible y alimenta una animación de Manim. Los resultados se contrastan con un oráculo independiente basado en eliminar vértices y contar componentes mediante BFS.

Motivación y definición

En una red de transporte o comunicaciones, una estación o nodo puede concentrar conexiones entre zonas. Si deja de operar, partes de la red pueden quedar incomunicadas. Identificar estos puntos permite analizar vulnerabilidades sin simular por separado cada fallo.

Sea $G = (V, E)$ un grafo no dirigido. Un vértice u es articulación si $c(G - u) > c(G)$, donde c es el número de componentes conexas. La definición también cubre grafos desconectados: la comparación se hace contra el número de componentes original.



Grafo del video. En dorado: articulaciones A, B, D, E. El ciclo $B - C - D - B$ muestra una ruta de retorno.

Idea de DFS

DFS construye un bosque de búsqueda. $tin[u]$ es el instante en que se descubre u. $low[u]$ es el menor tiempo alcanzable desde el subárbol de u usando aristas del árbol y, como máximo, una arista de retorno a un ancestro.

Reglas del algoritmo

Al terminar un hijo DFS v de u, la información de v permite decidir si el subárbol puede volver a un ancestro propio de u:

$$low[u] = \text{mín}\,(tin[u],\ tin[w] \text{ de retornos a ancestros},\ low[v] \text{ de hijos})\,.$$

Para un vértice no raíz, u es articulación si existe un hijo v con $low[v] \geq tin[u]$. La igualdad importa: si $low[v] = tin[u]$, el subárbol solo retorna a u, y al retirar u queda separado. La raíz se trata aparte: es articulación si tiene más de un hijo en el bosque DFS.

```
dfs(u):
  tin[u] = low[u] = reloj; reloj += 1
  para v en vecinos_ordenados(u):
    si v es padre[u]: continuar
    si v no visitado:
      padre[v] = u; hijos[u] += 1; dfs(v)
      low[u] = min(low[u], low[v])
      si u no es raiz y low[v] >= tin[u]: marcar u
    si no, si v es ancestro de u:
      low[u] = min(low[u], tin[v])
  si u es raiz y hijos[u] > 1: marcar u
```

Correctitud, en breve. Todo camino alternativo que sale del subárbol y evita a u debe usar una arista de retorno a un ancestro anterior a u. $low[v] < tin[u]$ certifica esa salida; si no se cumple, quitar u separa el subárbol hijo. Para la raíz no existe ancestro anterior, así que la desconexión se determina contando sus hijos DFS.

Complejidad. Con listas de adyacencia y vecinos visitados en tiempo constante, cada vértice se descubre una vez y cada arista se examina desde sus dos extremos: tiempo $O(V + E)$. El algoritmo usa $O(V)$ espacio auxiliar para tiempos, padres, estado y pila recursiva; el grafo ocupa $O(V + E)$. La traza incremental tiene $O(V + E)$ eventos.

Implementación, eventos y validación

El código separa el modelo del grafo, Tarjan, el registrador de eventos, la validación de trazas y la animación. La ejecución algorítmica no importa Manim. La primera versión acepta grafos simples no dirigidos con etiquetas de texto, incluye componentes desconectadas y rechaza lazos y aristas paralelas. El orden lexicográfico de vecinos hace reproducible la DFS.

Los eventos tienen número de secuencia, tipo, vértice, arista, padre, candidato y valores locales de *tin* y *low*. Se registran descubrimientos, exploración y clasificación de aristas, observaciones de retorno, actualizaciones, finalizaciones y articulaciones. La escena reproduce esta salida y un validador comprueba la secuencia y sus transiciones sin ejecutar Tarjan de nuevo.

Grafo ilustrativo. Se usan siete vértices y siete aristas: $A - B$, $A - G$, $B - C$, $C - D$, $D - B$, $D - E$ y $E - F$. Con exploración alfabética, los resultados son:

Vértice	A	B	C	D	E	F	G
tin	0	1	2	3	4	5	6
low	0	1	1	1	4	5	6

La arista de retorno $D - B$ propaga $low[D] = 1$ y $low[C] = 1$. Así, $low[C] = tin[B]$ satisface la condición de articulación para B. D y E cumplen la regla no raíz; A es raíz con dos hijos DFS. Resultado: $\{A, B, D, E\}$.

Pruebas. Quince pruebas automáticas cubren grafo vacío, vértice aislado, arista, camino, ciclo, estrella, desconectado, ausencia y multiplicidad de articulaciones, las dos condiciones de raíz, determinismo y trazas alteradas. Además, 160 grafos aleatorios reproducibles se comparan con un oráculo BFS que elimina cada vértice y vuelve a contar componentes; no reutiliza Tarjan.

Animación y reproducción

La película de Manim contiene siete secuencias: motivación con una estación fuera de servicio; contraste entre ciclo y puente; intuición de DFS, *tin* y *low*; ejecución sincronizada con eventos reales; caso especial de la raíz; complejidad; y conclusión con créditos. El render de producción dura 3:29, en formato horizontal 16:9 a 1280x720 y 30 FPS. El archivo se decodificó completo y se inspeccionaron cuadros de todas las secuencias, incluida la fórmula de *low* y los créditos.

Entorno comprobado: Windows; Python 3.11.9; Manim Community 0.21.0; Cairo; PyAV 18.1.0; LaTeX y dvisvgm. El ejecutable FFmpeg no está en PATH, pero el render de producción se completó con PyAV.

Reproducción desde la raíz del proyecto:

```
python scripts/run_example.py
manim render -qm --disable_caching --progress_bar none \\
  --media_dir media/final src/animation/articulation_film.py ArticulationFilm
```

El ejemplo imprime las articulaciones y los valores *tin* y *low* obtenidos por la implementación. Para preparar el entorno, reproducir las escenas o modificar el grafo, consulte README.md.

Reproducibilidad. El algoritmo se puede ejecutar independientemente de Manim mediante el ejemplo incluido en el repositorio. La película consume los eventos validados de una ejecución real de Tarjan; no almacena un recorrido prefijado.

Video final: Ver ArticulationFilm.mp4 en GitHub.